STEM BUILDERS

Conditional Statements & Loops

4. Conditional Statements & Loops

By allowing you to respond selectively to different situations and conditions, if statements open up whole new possibilities for your programs. In this section, you will learn how to test for certain conditions, and then respond in appropriate ways to those conditions.

What is an *if* statement?

An *if* statement tests for a condition, and then responds to that condition. If the condition is true, then whatever action is listed next gets carried out. You can test for multiple conditions at the same time, and respond appropriately to each condition.

Example

Here is an example that shows a number of the desserts I like. It lists those desserts, but lets you know which one is my favorite.

```
# A list of desserts I like.
desserts = ['ice cream', 'chocolate', 'apple crisp', 'cookies']
favorite_dessert = 'apple crisp'
# Print the desserts out, but let everyone know my favorite dessert.
for dessert in desserts:
    if dessert == favorite_dessert:
        # This dessert is my favorite, let's let everyone know!
        print("%s is my favorite dessert!" % dessert.title())
    else:
        # I like these desserts, but they are not my favorite.
        print("I like %s." % dessert)
```

Output:

```
I like ice cream.
I like chocolate.
Apple Crisp is my favorite dessert!
I like cookies.
```

What happens in this program?

- The program starts out with a list of desserts, and one dessert is identified as a favorite.
- The for loop runs through all the desserts.
- Inside the for loop, each item in the list is tested.
  - If the current value of *dessert* is equal to the value of *favorite_dessert*, a message is printed that this is my favorite.
  - If the current value of *dessert* is not equal to the value of *favorite_dessert*, a message is printed that I just like the dessert.

You can test as many conditions as you want in an if statement, as you will see in a little bit.

Logical Tests

Every if statement evaluates to *True* or *False*. *True* and *False* are Python keywords, which have special meanings attached to them. You can test for the following conditions in your if statements:

- equality (==)
- inequality (!=)
- other inequalities
  - greater than (>)
  - greater than or equal to (>=)
  - less than (<)
  - less than or equal to (<=)
- You can test if an item is in a list.

Equality

Two items are *equal* if they have the same value. You can test for equality between numbers, strings, and a number of other objects which you will learn about later. Some of these results may be surprising, so take a careful look at the examples below.

In Python, as in many programming languages, two equals signs tests for equality.

Watch out! Be careful of accidentally using one equals sign, which can really throw things off because that one equals sign actually sets your item to the value you are testing for!

```
5 == 5
Output: True

3 == 5
Output: False

5 == 5.0
Output: True

'eric' == 'eric'
Output: True

'Eric' == 'eric'
Output: False

'Eric'.lower() == 'eric'.lower()
Output: True

'5' == 5
Output: False

'5' == str(5)
Output: True
```

Inequality

Two items are *inequal* if they do not have the same value. In Python, we test for inequality using the exclamation point and one equals sign.

Sometimes you want to test for equality and if that fails, assume inequality. Sometimes it makes more sense to test for inequality directly.

```
3 != 5
Output: True

5 != 5
Output: False

'Eric' != 'eric'
Output: True
```

Other Inequalities

greater than

```
5 > 3
```
Output: True

greater than or equal to

```
5 >= 3
```
Output: True

```
3 >= 3
```
Output: True

less than

```
3 < 5
```
Output: True

less than or equal to

```
3 <= 5
```
Output: True

```
3 <= 3
```
Output: True

Checking if an item is in a list

You can check if an item is in a list using the **in** keyword.

```
vowels = ['a', 'e', 'i', 'o', 'u'] 'a' in vowels
```
Output: True

```
vowels = ['a', 'e', 'i', 'o', 'u'] 'b' in vowels
```
Output: False

Exercises

True and False

- Write a program that consists of at least ten lines, each of which has a logical statement on it. The output of your program should be 5 **True**s and 5 **False**s.
- Note: You will probably need to write `print(5 > 3)`, not just `5 > 3`.

The if-elif...else chain

You can test whatever series of conditions you want to, and you can test your conditions in any combination you want.

Simple if statements

The simplest test has a single **if** statement, and a single statement to execute if the condition is **True**.

```
dogs = ['willie', 'hootz', 'peso', 'juno']
if len(dogs) > 3:
     print("Wow, we have a lot of dogs here!")
```

```
Output:
Wow, we have a lot of dogs here!
```

In this situation, nothing happens if the test does not pass.

```
dogs = ['willie', 'hootz']
if len(dogs) > 3:
     print("Wow, we have a lot of dogs here!")
```

Notice that there are no errors. The condition `len(dogs) > 3` evaluates to False, and the program moves on to any lines after the **if** block.

if-else statements

Many times you will want to respond in two possible ways to a test. If the test evaluates to **True**, you will want to do one thing. If the test evaluates to **False**, you will want to do something else. The **if-else** structure lets you do that easily. Here's what it looks like:

```
dogs = ['willie', 'hootz', 'peso', 'juno']
if len(dogs) > 3:
     print("Wow, we have a lot of dogs here!")
else:
     print("Okay, this is a reasonable number of dogs.")
```

Output:
```
Wow, we have a lot of dogs here!
```

Our results have not changed in this case, because if the test evaluates to **True** only the statements under the **if** statement are executed. The statements under **else** area only executed if the test fails:

```
dogs = ['willie', 'hootz']
 if len(dogs) > 3:
     print("Wow, we have a lot of dogs here!")
 else:
     print("Okay, this is a reasonable number of dogs.")
```

Output:
```
Okay, this is a reasonable number of dogs.
```

The test evaluated to **False**, so only the statement under `else` is run.

if-elif...else chains

Many times, you will want to test a series of conditions, rather than just an either-or situation. You can do this with a series of if-elif-else statements

There is no limit to how many conditions you can test. You always need one if statement to start the chain, and you can never have more than one else statement. But you can have as many elif statements as you want.

```
dogs = ['willie', 'hootz', 'peso', 'monty', 'juno', 'turkey']
  if len(dogs) >= 5:
     print("Holy mackerel, we might as well start a dog hostel!")
 elif len(dogs) >= 3:
     print("Wow, we have a lot of dogs here!")
 else:
```

```
    print("Okay, this is a reasonable number of dogs.")
```

Output:
```
Holy mackerel, we might as well start a dog hostel!
```

It is important to note that in situations like this, only the first test is evaluated. In an if-elif-else chain, once a test passes the rest of the conditions are ignored.

```
dogs = ['willie', 'hootz', 'peso', 'monty']
 if len(dogs) >= 5:
    print("Holy mackerel, we might as well start a dog hostel!")
 elif len(dogs) >= 3:
    print("Wow, we have a lot of dogs here!")
 else:
    print("Okay, this is a reasonable number of dogs.")
```

Output:
```
Wow, we have a lot of dogs here!
```

The first test failed, so Python evaluated the second test. That test passed, so the statement corresponding to `len(dogs) >= 3` is executed.

```
dogs = ['willie', 'hootz']
 if len(dogs) >= 5:
    print("Holy mackerel, we might as well start a dog hostel!")
 elif len(dogs) >= 3:
    print("Wow, we have a lot of dogs here!")
 else:
    print("Okay, this is a reasonable number of dogs.")
```

Output:
```
Okay, this is a reasonable number of dogs.
```

In this situation, the first two tests fail, so the statement in the else clause is executed. Note that this statement would be executed even if there are no dogs at all:

```
dogs = []
 if len(dogs) >= 5:
    print("Holy mackerel, we might as well start a dog hostel!")
 elif len(dogs) >= 3:
    print("Wow, we have a lot of dogs here!")
 else:
    print("Okay, this is a reasonable number of dogs.")
```

Output:
```
Okay, this is a reasonable number of dogs.
```

Note that you don't have to take any action at all when you start a series of if statements. You could simply do nothing in the situation that there are no dogs by replacing the `else` clause with another `elif` clause:

```
dogs = []
if len(dogs) >= 5:
     print("Holy mackerel, we might as well start a dog hostel!")
 elif len(dogs) >= 3:
     print("Wow, we have a lot of dogs here!")
elif len(dogs) >= 1:
    print("Okay, this is a reasonable number of dogs.")
```

In this case, we only print a message if there is at least one dog present. Of course, you could add a new `else` clause to respond to the situation in which there are no dogs at all:

```
dogs = []
if len(dogs) >= 5:
     print("Holy mackerel, we might as well start a dog hostel!")
elif len(dogs) >= 3:
     print("Wow, we have a lot of dogs here!")
elif len(dogs) >= 1:
     print("Okay, this is a reasonable number of dogs.")
else:
    print("I wish we had a dog here.")
```

Output:

```
I wish we had a dog here.
```

As you can see, the if-elif-else chain lets you respond in very specific ways to any given situation.

Exercises

Three is a Crowd

- Make a list of names that includes at least four people.
- Write an if test that prints a message about the room being crowded, if there are more than three people in your list.
- Modify your list so that there are only two people in it. Use one of the methods for removing people from the list, don't just redefine the list.
- Run your if test again. There should be no output this time, because there are less than three people in the list.
- **Bonus:** Store your if test in a function called something like `crowd_test`.

Three is a Crowd - Part 2

- Save your program from *Three is a Crowd* under a new name.
- Add an `else` statement to your if tests. If the `else` statement is run, have it print a message that the room is not very crowded.

Six is a Mob

- Save your program from *Three is a Crowd - Part 2* under a new name.
- Add some names to your list, so that there are at least six people in the list.
- Modify your tests so that
  - If there are more than 5 people, a message is printed about there being a mob in the room.
  - If there are 3-5 people, a message is printed about the room being crowded.
  - If there are 1 or 2 people, a message is printed about the room not being crowded.
  - If there are no people in the room, a message is printed about the room being empty.

More than one passing test

In all of the examples we have seen so far, only one test can pass. As soon as the first test passes, the rest of the tests are ignored. This is really good, because it allows our code to run more efficiently. Many times only one condition can be true, so testing every condition after one passes would be meaningless.

There are situations in which you want to run a series of tests, where every single test runs. These are situations where any or all of the tests could pass, and you want to respond to each passing test. Consider the following example, where we want to greet each dog that is present:

```
dogs = ['willie', 'hootz']
if 'willie' in dogs:
     print("Hello, Willie!")
if 'hootz' in dogs:
     print("Hello, Hootz!")
if 'peso' in dogs:
     print("Hello, Peso!")
if 'monty' in dogs:
     print("Hello, Monty!")
```

```
Output:
Hello, Willie!
Hello, Hootz!
```

If we had done this using an if-elif-else chain, only the first dog that is present would be greeted:

```
dogs = ['willie', 'hootz']
if 'willie' in dogs:
     print("Hello, Willie!")
elif 'hootz' in dogs:
    print("Hello, Hootz!")
elif 'peso' in dogs:
    print("Hello, Peso!")
elif 'monty' in dogs:
    print("Hello, Monty!")
```

Output:
```
Hello, Willie!
```

Of course, this could be written much more cleanly using lists and for loops. See if you can follow this code.

```
dogs_we_know = ['willie', 'hootz', 'peso', 'monty', 'juno', 'turkey']
dogs_present = ['willie', 'hootz']
# Go through all the dogs that are present, and greet the dogs we know.
for dog in dogs_present:
     if dog in dogs_we_know:
         print("Hello, %s!" % dog.title())
```

Output:
```
Hello, Willie!
Hello, Hootz!
```

This is the kind of code you should be aiming to write. It is fine to come up with code that is less efficient at first. When you notice yourself writing the same kind of code repeatedly in one program, look to see if you can use a loop or a function to make your code more efficient.

True and False values

Every value can be evaluated as True or False. The general rule is that any non-zero or non-empty value will evaluate to True. If you are ever unsure, you can open a Python terminal and write two lines to find out if the value you are considering is True or False. Take a look at the following examples, keep them in mind, and test any value you are curious about. I am using a slightly longer test just to make sure something gets printed each time.

```
if 0:
    print("This evaluates to True.")
else:
     print("This evaluates to False.")
```

Output:
```
This evaluates to False.
```

```
if 1:
    print("This evaluates to True.")
else:
     print("This evaluates to False.")
```

Output:
```
This evaluates to True.
```

```
# Arbitrary non-zero numbers evaluate to True.
if 1253756:
    print("This evaluates to True.")
else:
     print("This evaluates to False.")
```

Output:
```
This evaluates to True.
```

```
# Negative numbers are not zero, so they evaluate to True.
if -1:
    print("This evaluates to True.")
else:
     print("This evaluates to False.")
```

Output:
```
This evaluates to True.
```

```
# An empty string evaluates to False.
if '':
    print("This evaluates to True.")
else:
     print("This evaluates to False.")
```

Output:
This evaluates to False.

```
# Any other string, including a space, evaluates to True.
if ' ':
    print("This evaluates to True.")
else:
     print("This evaluates to False.")
```

Output:
This evaluates to True.

```
# Any other string, including a space, evaluates to True.
if 'hello':
    print("This evaluates to True.")
else:
     print("This evaluates to False.")
```

Output:
This evaluates to True.

```
# None is a special object in Python. It evaluates to False.
if None:
    print("This evaluates to True.")
else:
     print("This evaluates to False.")
```

Output:
This evaluates to False.

Overall Challenges

Celsius to Fahrenheit Calculator:

Write a Python program to convert temperatures to and from celsius, fahrenheit.
[Formula : c/5 = f-32/9 [where c = temperature in celsius and f = temperature in fahrenheit]
Expected Output :
60°C is 140 in Fahrenheit
45°F is 7 in Celsius

Count Even and Odd numbers:

Write a Python program to count the number of even and odd numbers from a series of numbers. Go to the editor
Sample numbers : numbers = (1, 2, 3, 4, 5, 6, 7, 8, 9)
Expected Output :
Number of even numbers : 5
Number of odd numbers : 4

Motivate to Innovate...

STEM BUILDERS Learning Center

MATH | ROBOTICS | TECHNOLOGY | EVENTS | CAMPS

contactus@stembuilders.com

www.stembuilders.com